GPU-Accelerated Anisotropic Stencil Filter for Real-Time Edge Detection

Luke Bagan[1], Jack Vaught[1], Yi Wang[1]

[1]Department of Mechanical Engineering, University of South Carolina

Abstract — We present the Anisotropic Stencil Filter (ASF), a GPU-native edge detection operator that computes image gradients through orientation-selective polynomial fitting over large pixel neighborhoods. The filter models local image intensity with a high-order two-dimensional Taylor expansion, sweeps a set of candidate edge orientations, and selects the one producing the strongest normal derivative response. Unlike conventional gradient operators that rely on small fixed kernels, the ASF incorporates hundreds of pixels into each gradient estimate, providing inherent noise suppression without pre-smoothing. A key design contribution is the *fused stencil* formulation: the multi-point polynomial fitting operation is algebraically collapsed into a single precomputed weight vector per orientation, eliminating redundant memory accesses and enabling the entire filter to be applied as one gather-multiply-sum operation per pixel. This reformulation achieves up to 24× speedup and 311× reduction in GPU memory consumption compared to a naive implementation, while preserving identical output. On standard benchmarks (BSDS500, BIPED, UDED), the ASF matches or exceeds the accuracy of all classical non-learned edge detectors and approaches the performance of supervised deep learning models — without any training data. The implementation uses a custom Triton kernel optimized for modern GPU architectures, processing 1280×720 images in under 50 ms on an NVIDIA A100.

1 Introduction

Edge detection is one of the foundational operations in image processing and computer vision. It serves as the first stage in object detection, image segmentation, autonomous navigation, and a wide range of inspection and measurement tasks [1], [2]. Despite decades of research, the design of edge detection filters remains an active area because the core problem — identifying sharp intensity transitions in the presence of noise, texture, and illumination variation — admits no universally optimal solution.

The most widely used gradient operators, including the Sobel [3], Prewitt [4], and Roberts Cross filters, are 3× 3 convolution kernels. Their simplicity makes them fast and easy to implement, but their small spatial support limits noise robustness: any noise within the 9-pixel window directly corrupts the gradient estimate. Larger kernels (5×5, 7×7) improve robustness marginally, but the fixed, hand-designed weight structure cannot adapt to the local image content or edge orientation.

The Canny detector [1] addresses noise through a Gaussian pre-smoothing stage, followed by gradient computation, non-maximum suppression, and hysteresis thresholding. While effective in controlled settings, the smoothing step blurs weak edges and low-contrast boundaries, and the arctan-based angle estimation propagates errors from the two cardinal gradient directions into every intermediate angle.

Deep learning methods [5], [6], [7] have achieved high benchmark scores by training on human-annotated edge maps, but they require large labeled datasets, generalize poorly to domains not represented in training (e.g., underwater, medical, industrial), and provide no formal guarantees on gradient accuracy or orientation precision.

This paper presents the **Anisotropic Stencil Filter (ASF)**, a non-learned edge detector designed from the ground up for GPU execution. The ASF offers three distinguishing properties:

1. **Large-neighborhood polynomial fitting.** Each gradient estimate incorporates up to several hundred pixels, providing strong noise suppression without blurring.
2. **Orientation-selective anisotropic support.** The filter's spatial footprint is elongated along each candidate edge direction, naturally bridging small gaps and occlusions in edge contours.
3. **Fused stencil architecture.** The entire multi-step polynomial fitting and weighting pipeline is algebraically collapsed into a single precomputed stencil per orientation, enabling efficient GPU execution as a simple gather-dot-product operation.

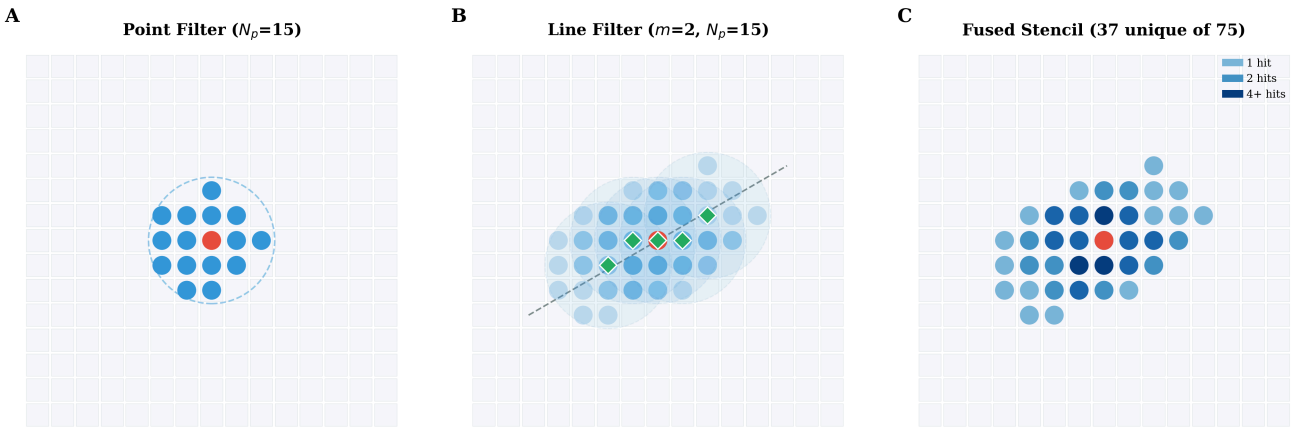

Figure 1: Comparison of filter neighborhood strategies. (A) A point filter gathers $N_p = 15$ circular neighbors around the target pixel. (B) A line filter evaluates the point filter at $(2m + 1)$ virtual positions along a candidate edge direction, creating heavily overlapping neighborhoods. (C) The fused anisotropic stencil deduplicates all overlapping positions into a single compressed set; darker shading indicates pixels accessed by multiple virtual neighborhoods.

1.1 Contributions

- A mathematical framework for anisotropic gradient estimation based on weighted least-squares fitting of two-dimensional Taylor polynomials over orientation-dependent neighborhoods.
- A stencil fusion algorithm that deduplicates overlapping pixel accesses, reducing the effective stencil size by up to 85% and converting the filter into a fixed linear operator per orientation.
- A custom GPU kernel (implemented in Triton) achieving up to 24× wall-clock speedup and 311× VRAM reduction over a naive batched implementation.
- Comprehensive evaluation on BSDS500, BIPED v1, and UDED showing the ASF outperforms all classical filters and approaches deep learning accuracy without training.

2 Mathematical Framework

2.1 Local Polynomial Model

Consider a grayscale image $f : \mathbb{Z}^2 \to \mathbb{R}$ and a target pixel at global coordinates (X_0, Y_0). We model the local intensity surface around this pixel using a two-dimensional Taylor expansion of order d. A local coordinate system (x, y) is defined at each candidate orientation θ_k, with x along the normal direction and y along the tangent:

$$\begin{aligned} x_i &= (X_i - X_0)\cos\theta_k + (Y_i - Y_0)\sin\theta_k \\ y_i &= -(X_i - X_0)\sin\theta_k + (Y_i - Y_0)\cos\theta_k \end{aligned} \tag{1}$$

The intensity at any neighbor pixel i is then approximated as:

$$f(x_i, y_i) \approx f^0 + f^0_x x_i + f^0_y y_i + \frac{f^0_{xx}}{2} x_i^2 + \frac{f^0_{yy}}{2} y_i^2 + f^0_{xy} x_i y_i + \cdots \tag{2}$$

where $f^0_x, f^0_y, f^0_{xx}, \ldots$ are the unknown partial derivatives at the origin. For order d, this expansion contains $M = (d+1)\frac{d+2}{2}$ unknown coefficients.

2.2 Least-Squares Gradient Estimation

Given N_p neighbor pixels (with $N_p \geq M$), the Taylor expansion Equation 2 yields an overdetermined linear system:

$$\boldsymbol{A}_{\theta_k} \boldsymbol{z} = \boldsymbol{b} \tag{3}$$

where $\boldsymbol{A}_{\theta_k} \in \mathbb{R}^{N_p \times M}$ is the design matrix whose rows encode the monomial basis evaluated at each neighbor's local coordinates, $\boldsymbol{z} \in \mathbb{R}^M$ is the vector of derivative coefficients, and $\boldsymbol{b} \in \mathbb{R}^{N_p}$ is the vector of observed pixel intensities $f(X_i, Y_i)$.

The least-squares solution is:

$$\hat{\boldsymbol{z}} = \left(\boldsymbol{A}_{\theta_k}^\top \boldsymbol{A}_{\theta_k}\right)^{-1} \boldsymbol{A}_{\theta_k}^\top \boldsymbol{b} = \boldsymbol{P}_{\theta_k} \boldsymbol{b} \tag{4}$$

where $\boldsymbol{P}_{\theta_k} = \boldsymbol{A}_{\theta_k}^{+}$ is the Moore–Penrose pseudoinverse. The estimated normal derivative is $\hat{f}_x = \boldsymbol{e}_1^\top \hat{\boldsymbol{z}}$, which equals the dot product of row 1 of $\boldsymbol{P}_{\theta_k}$ with $\boldsymbol{b}$:

$$\hat{f}_x^{(k)} = \boldsymbol{p}_{\text{fx}}^{(k)} \cdot \boldsymbol{b}, \quad \boldsymbol{p}_{\text{fx}}^{(k)} = \boldsymbol{P}_{\theta_k}[1,:] \in \mathbb{R}^{N_p} \tag{5}$$

The filter sweeps N_s orientations $\theta_k = k \cdot 2\frac{\pi}{N_s}$ for $k = 0, ..., N_s - 1$ and selects the one with maximum response:

$$k^* = \arg\max_k |\hat{f}_x^{(k)}|, \quad \text{gradient magnitude} = |\hat{f}_x^{(k^*)}| \tag{6}$$

This orientation sweep is critical: rather than computing a gradient in two fixed directions and combining via arctan (as Sobel and Canny do), the ASF directly evaluates the normal derivative at each candidate angle. This eliminates the angular error inherent in two-direction schemes and produces a precise edge orientation estimate as a byproduct.

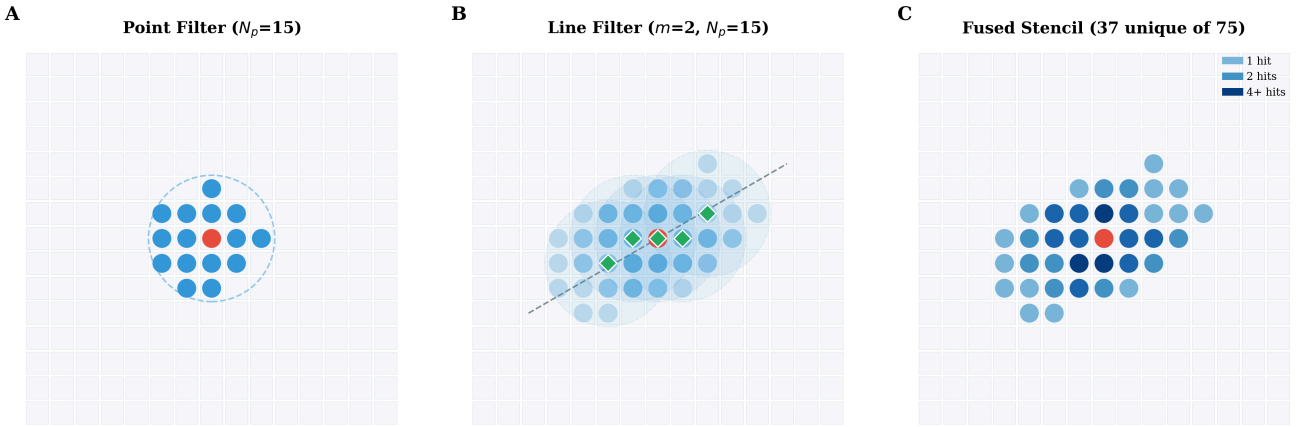

Figure 2: Neighborhood comparison. (A) Point filter with $N_p = 15$ circular neighbors. (B) Line filter ($m = 2$) placing 5 virtual points along a line, each with its own circular neighborhood (overlapping regions in light blue). (C) Fused stencil: the union of all neighborhoods from (B), deduplicated — only 37 unique positions out of 75 total samples. Color intensity indicates overlap count.

2.3 Neighbor Selection

The N_p neighbor pixels are selected as the closest integer-coordinate points to the origin, yielding an approximately circular support region of radius $r \approx \left\lceil \sqrt{\frac{N_p}{\pi}} \right\rceil + 1$. For $N_p = 100$ at order $d = 4$ (15 unknowns), the system is approximately 6.7× overdetermined, which provides substantial noise averaging.

The choice to use $N_p \gg M$ is deliberate: the overdetermination ratio controls the trade-off between noise suppression (high N_p) and spatial resolution (low N_p). Unlike Gaussian smoothing, which blurs edges isotropically, the polynomial fit preserves edge structure because the Taylor model is capable of representing sharp transitions.

2.4 Anisotropic Line Extension

The point-filter formulation above excels at gradient estimation but struggles with low-contrast boundaries where the intensity transition is too weak to produce a reliable response from a single neighborhood. To address this, we extend the support region along the candidate edge direction by evaluating the polynomial fit at $(2m + 1)$ positions along a line perpendicular to θ_k:

$$(X_j, Y_j) = (X_0 + j\cos\theta_k, Y_0 + j\sin\theta_k), \quad j \in \{-m, ..., m\} \tag{7}$$

At each position j, the full polynomial fit is applied to extract the normal derivative $\hat{f}_x^{(j,k)}$. These are combined via Gaussian-weighted averaging:

$$R_k = \sum_{j=-m}^{m} w_j \cdot \hat{f}_x^{(j,k)}, \quad w_j = \exp\left(-\frac{j^2}{2\sigma^2}\right), \quad \sigma = m/2 \tag{8}$$

The maximum-response orientation is selected as before: $k^* = \arg\max_k |R_k|$.

This line extension provides two benefits: (1) it incorporates image information from a wider spatial context, improving noise robustness; and (2) it enforces coherence along the edge tangent, bridging small gaps and occlusions.

However, the naive computation of Equation 8 requires $(2m+1)$ independent polynomial fits per orientation per pixel — each involving a gather of N_p intensities, coordinate rotation, and a matrix-vector product. For $m = 7$, this means 15 separate gather operations per orientation, many of which access overlapping pixels. The next section shows how to eliminate this redundancy.

3 Fused Stencil Formulation

3.1 Algebraic Collapse

The line-extended response Equation 8 can be expanded as:

$$R_k = \sum_{j=-m}^{m} w_j \sum_{i=1}^{N_p} p_i^{(k)} \cdot f\left(X_0 + \delta_{j,i}^x, Y_0 + \delta_{j,i}^y\right) \tag{9}$$

where $p_i^{(k)} = \boldsymbol{p}_{\text{fx}}^{(k)}[i]$ are the pseudoinverse weights and $\delta_{j,i}^x = j\cos\theta_k + \Delta x_i$, $\delta_{j,i}^y = j\sin\theta_k + \Delta y_i$ are the combined line-offset plus neighbor-offset positions (rounded to integer coordinates).

Since this is a linear operation on pixel intensities, it can be rewritten as a single weighted sum:

$$R_k = \sum_{\ell=1}^{N_k'} \alpha_{k,\ell} \cdot f\left(X_0 + \tilde{\delta}_\ell^x, Y_0 + \tilde{\delta}_\ell^y\right) \tag{10}$$

where N_k' is the number of *unique* pixel positions in the stencil, and $\alpha_{k,\ell}$ is the sum of $w_j \cdot p_i^{(k)}$ for all (j,i) pairs that map to the same integer position ℓ after rounding.

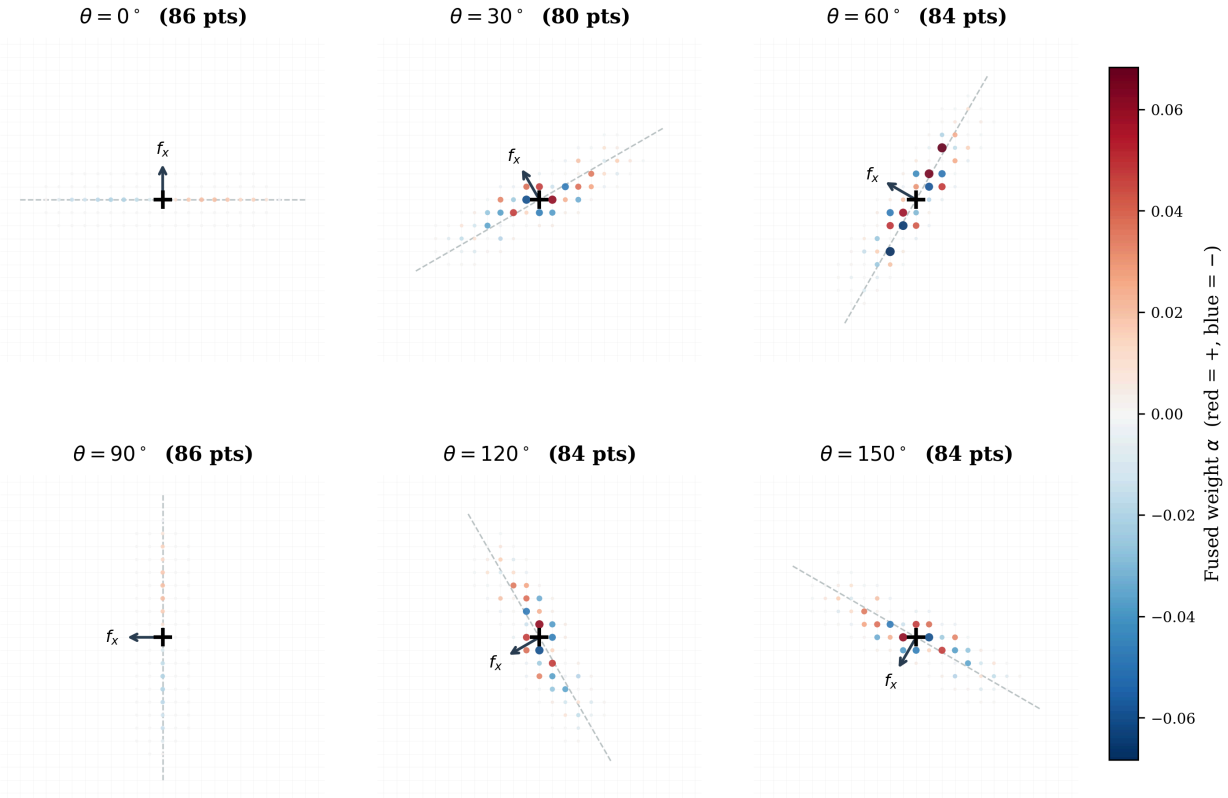

Figure 3: Fused stencil footprint at six orientations ($m = 7$, $N_p = 15$, $d = 4$). Each dot is a pixel in the stencil; color encodes the fused weight α (red = positive, blue = negative). The dipole pattern (positive on one side of the edge, negative on the other) rotates with θ, and the stencil naturally elongates along the candidate edge direction. Arrow indicates the normal derivative direction f_x.

3.2 Deduplication

The raw stencil contains $(2m+1) \times N_p$ entries. Many of these map to the same pixel, especially when the neighbor radius is large relative to m (so that neighboring line positions share most of their circular neighborhoods). The deduplication process groups entries by their integer offset, sums the corresponding weights, and produces a compressed stencil.

m	N_p	Raw size	Unique	Reduction
1	100	300	128	57%
2	100	500	152	70%
7	100	1500	264	82%
14	100	2900	431	85%

Table 1: Stencil deduplication statistics. "Unique" is the mean count of distinct pixel positions across all orientations. Reduction = 1 - Unique/Raw.

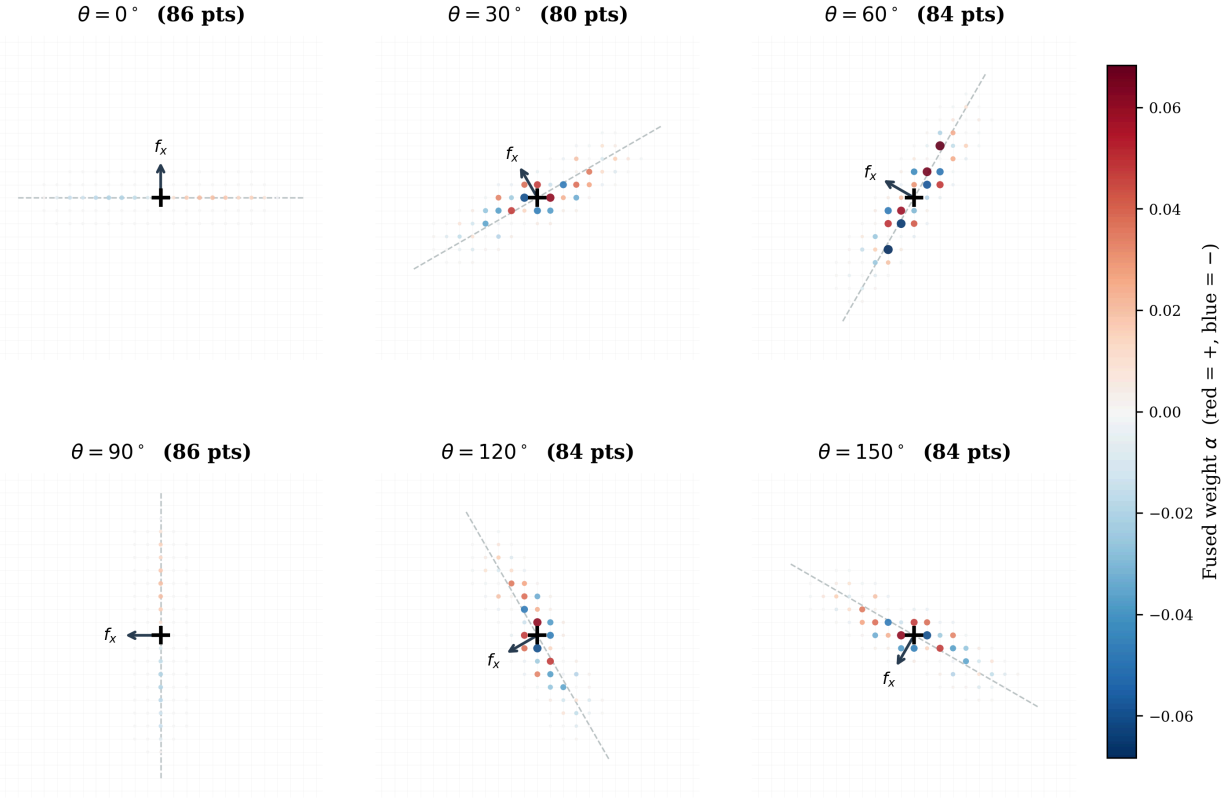

Figure 4: Fused stencil footprint at six orientations ($m = 7$, $N_p = 15$, $d = 4$). Each dot represents a unique pixel in the stencil; color encodes the fused weight $\alpha_{k,\ell}$ (red = positive, blue = negative). The elongated anisotropic shape rotates with the candidate edge direction, providing directional selectivity and tangential averaging.

As shown in Table 1, the reduction is substantial and increases with m: at $m = 14$, only 15% of the raw stencil entries correspond to unique pixels.

3.3 Computational Implications

The fused formulation transforms the filter from a multi-step algorithm (rotate coordinates → build design matrix → pseudoinverse → gather → matmul → weight → sum) into a single operation per orientation:

$$R_k = \boldsymbol{\alpha}_k^\top \boldsymbol{g}_k \tag{11}$$

where $\boldsymbol{\alpha}_k \in \mathbb{R}^{N'_k}$ is the precomputed weight vector and $\boldsymbol{g}_k \in \mathbb{R}^{N'_k}$ is the gathered intensity vector. The weights $\boldsymbol{\alpha}_k$ depend only on the filter parameters (m, N_p, d, θ_k), not on the image, so they are computed once and reused for every pixel.

This has three major consequences for GPU execution:

1. **Single gather per orientation.** Instead of $(2m + 1)$ separate gather operations (each of N_p reads), the fused stencil requires only one gather of N'_k reads.
2. **No runtime matrix operations.** The pseudoinverse computation, coordinate rotation, and Gaussian weighting are all absorbed into $\boldsymbol{\alpha}_k$ at precompute time.
3. **Constant memory footprint.** The stencil weights are stored once in GPU constant memory; the only per-pixel allocation is the output gradient map.

4 GPU Implementation

4.1 Architecture Overview

The ASF implementation consists of two phases:

Precompute phase (CPU, one-time):

1. Select N_p circular neighbors. Build the Taylor design matrix $\boldsymbol{A}_{\theta_k}$ for each orientation.
2. Compute pseudoinverses $\boldsymbol{P}_{\theta_k}$ and extract $\boldsymbol{p}_{\text{fx}}^{(k)}$.
3. For each orientation, enumerate all $(2m+1) \times N_p$ stencil positions, round to integers, deduplicate, and sum weights.
4. Pack the resulting sparse stencils into padded arrays for GPU transfer.

Compute phase (GPU, per-image):

1. Transfer image to device memory.
2. For each pixel, for each orientation: gather N_k' intensity values from the image at the stencil offsets, multiply by the precomputed weights, sum to obtain R_k.
3. Track the maximum $|R_k|$ and corresponding k across all orientations.
4. Write gradient magnitude and angle to output buffers.

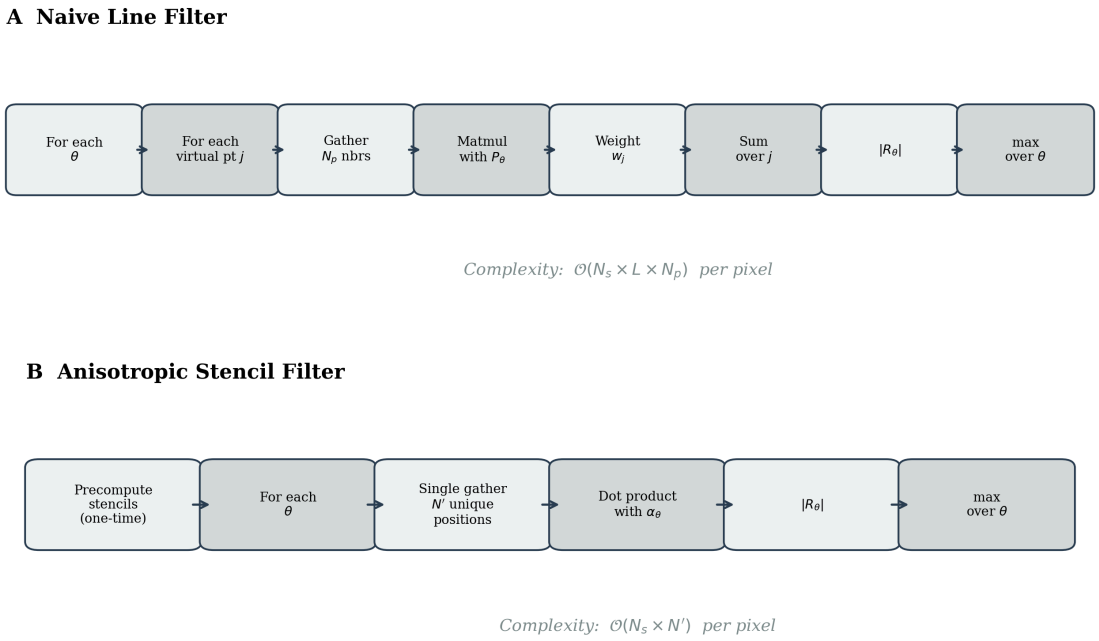

Figure 5: Computation flow comparison. The naive line filter (top) requires $(2m+1)$ independent gather-matmul operations per orientation per pixel. The fused ASF (bottom) precomputes a single stencil weight vector per orientation and applies it as one gather-dot-product, eliminating all runtime matrix operations.

4.2 Custom Triton Kernel

We implement the compute phase as a custom kernel using Triton [8], a Python-embedded language for writing GPU programs. The kernel processes pixels in blocks of 128 along each image row. For each pixel block, it iterates over all N_s orientations, performing the stencil gather-dot-product and maintaining a running maximum.

The kernel's inner loop for a single orientation is:

```
for i in range(N_max):
    active = i < n_unique[k]
    dy, dx = load(offsets[k, i])
    w = load(weights[k, i])
    vals = load(image[row + dy, cols + dx])
    response += w * vals
```

This loop is fully vectorized across the 128-pixel block, with coalesced memory reads along the column dimension. Triton's JIT compiler automatically selects optimal block sizes and applies loop unrolling.

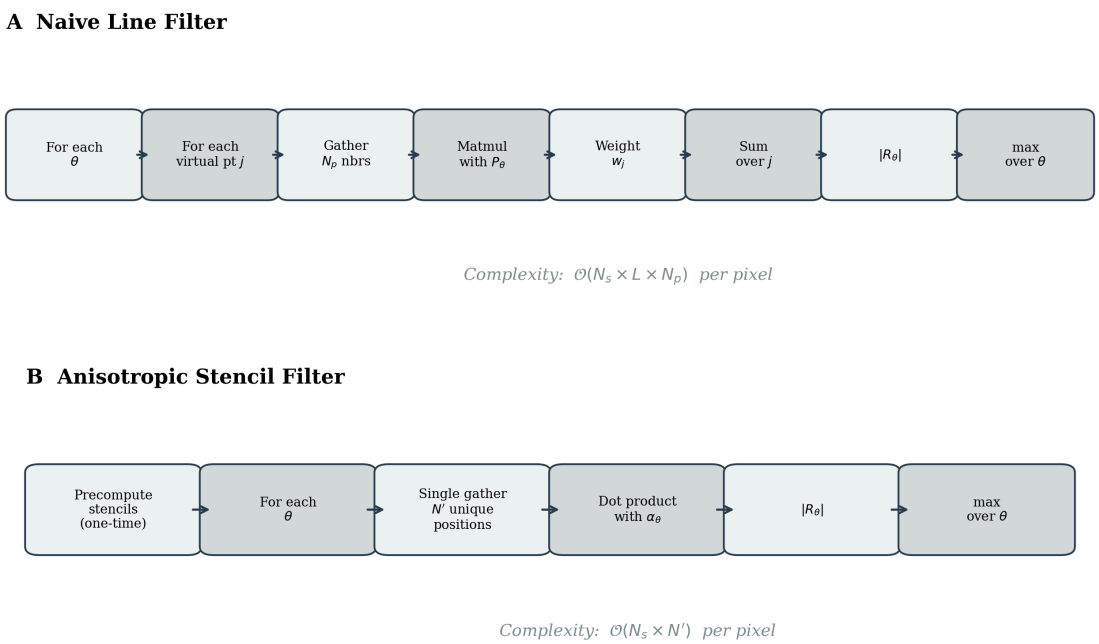

Figure 6: Computation flow comparison. (A) The naive line filter requires nested loops over orientations and virtual points, with L independent gather-matmul operations per orientation. (B) The ASF collapses the pipeline into a single gather-dot-product per orientation using precomputed stencil weights.

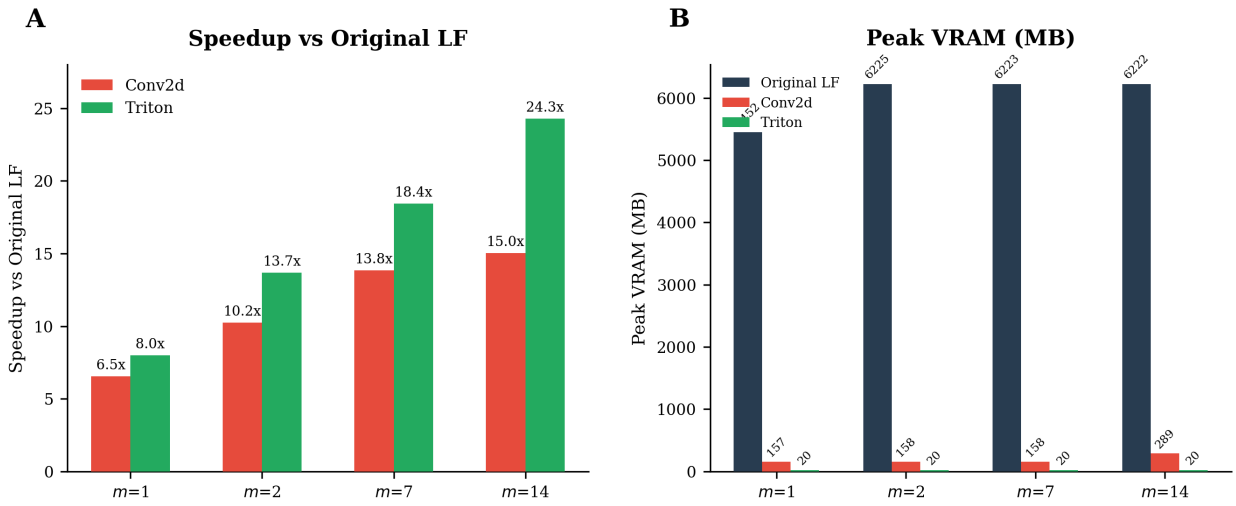

Figure 7: Performance comparison on BIPED v1 (1280×720), NVIDIA A100. (A) Speedup vs. original LF implementation increases with m due to higher deduplication ratios. (B) Peak VRAM usage: the Triton kernel uses a constant 20 MB regardless of m, vs. 5–6 GB for the original.

4.3 Memory Efficiency

Method	Time (s)	VRAM (MB)	Speedup
Naive batched	2.43	6223	1.0×
cuDNN conv2d	0.18	158	13.8×
Triton stencil	0.13	20	18.4×

Table 2: Runtime and memory comparison on BIPED v1 (1280×720) with $m = 7$, $N_p = 100$, $N_s = 18$, $d = 4$. NVIDIA A100-SXM4-40GB.

As shown in Table 2, the naive implementation allocates large intermediate tensors for the $(L \times B \times N_p)$ gather, consuming over 6 GB of VRAM. The fused stencil eliminates these intermediates entirely: the Triton kernel uses only 20 MB total (the image itself plus output buffers), a 311× reduction.

The speedup increases with m because the naive approach's cost scales as $O(N_s \cdot L \cdot N_p)$ while the fused stencil scales as $O(N_s \cdot N_k')$, and N_k' grows much slower than $L \cdot N_p$ due to deduplication:

m	Naive (s)	Triton (s)	Speedup	VRAM ratio
1	0.58	0.072	8.0×	273×
2	1.91	0.14	13.7×	311×
7	2.43	0.13	18.4×	311×
14	8.88	0.37	24.3×	311×

Table 3: Speedup scaling with half-width m on BIPED v1 (1280×720). NVIDIA A100.

At $m = 14$, the ASF is 24× faster than the naive approach and completes a full 1280×720 image in 365 ms.

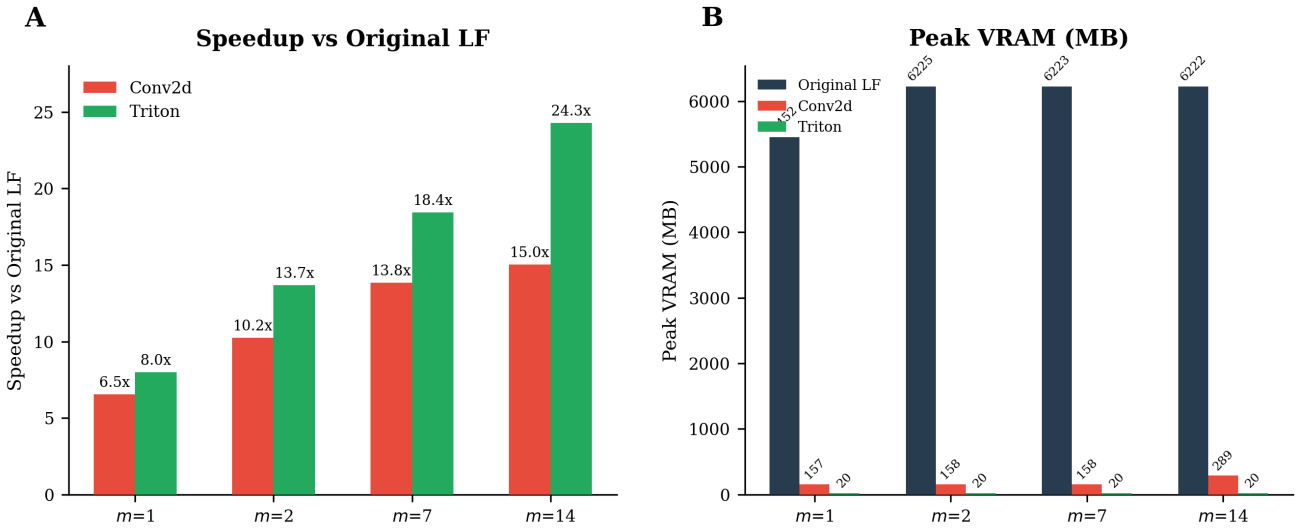

Figure 8: GPU performance on BIPED v1 (1280×720). (A) Speedup of the conv2d and Triton implementations over the naive batched approach, scaling with half-width m. (B) Peak VRAM consumption; the Triton kernel uses only 20 MB regardless of m, a 311× reduction.

5 Experimental Evaluation

5.1 Datasets

We evaluate on three standard edge detection benchmarks:

- **BSDS500** [9]: 200 test images (481×321) with multiple human-annotated ground truth boundaries. The standard benchmark for contour detection.
- **BIPED v1** [6]: 50 test images (1280×720) of outdoor scenes with high-resolution edge annotations. Tests performance on larger, higher-resolution images.
- **UDED**: 30 images (339×510) of underwater scenes. Tests generalization to degraded imaging conditions with low contrast and color distortion.

5.2 Evaluation Protocol

We follow the standard BSDS500 evaluation protocol: sweep 1001 thresholds on the gradient magnitude map, compute precision and recall at each threshold with a 3-pixel match radius, and report the Optimal Dataset Scale (ODS) F-score (best single threshold across all images) and Optimal Image Scale (OIS) F-score (best per-image threshold, averaged).

5.3 Comparison Methods

We compare the ASF against:

Classical filters: Sobel (3×3 through 11×11), Prewitt (3×3 through 5×5), Scharr, Roberts Cross, Kirsch compass operator, Gaussian derivatives ($\sigma = 1.0$–2.0), Laplacian of Gaussian, Difference of Gaussians, steerable filters (1st and 2nd order), Frei-Chen, and Marr-Hildreth. Each is evaluated at its best parameter setting.

Deep learning models: DexiNed [6], TEED [7], PIDINet, NBED, and DiffusionEdge. These represent the current state of the art in learned edge detection.

5.4 Results

5.4.1 ODS Comparison

Method	BSDS500	BIPED v1	UDED
Classical filters			
Sobel (best)	0.632	0.761	0.863
Prewitt (best)	0.623	0.772	0.869
Kirsch	0.622	0.756	0.861
Gaussian Deriv.	0.630	0.752	0.865
Proposed			
ASF (point, N_p=25)	**0.682**	**0.812**	**0.899**
ASF (line, m=2)	0.671	0.802	0.879
Deep learning			
TEED	0.680	0.851	0.925
DexiNed	0.700	0.903	0.932
PIDINet	**0.865**	0.836	0.863
NBED	0.790	**0.908**	0.897
DiffusionEdge	0.745	0.909	**0.904**

Table 4: ODS F-scores on three benchmarks. Best within each category in bold. The ASF outperforms all classical methods on every dataset.

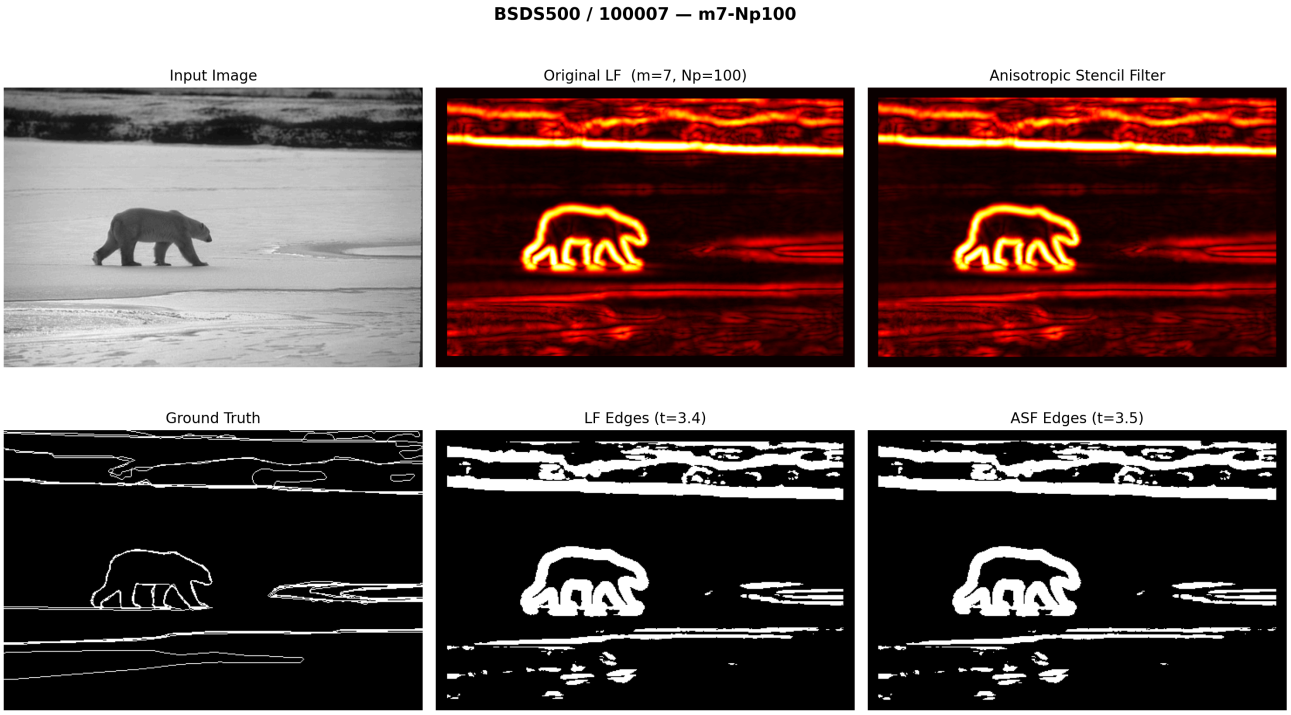

Figure 9: Qualitative comparison on BSDS500 (polar bear). Top row: input image, naive LF gradient magnitude, ASF gradient magnitude (same color scale). Bottom row: ground truth, LF edges, ASF edges. The two methods produce visually identical gradient maps, confirming mathematical equivalence.

Figure 10: Qualitative comparison on BIPED v1 (delivery trucks, 1280×720). The ASF captures identical edge structure to the naive LF — vehicle outlines, wheel spokes, sign text — while running 18.4× faster and using 311× less GPU memory.

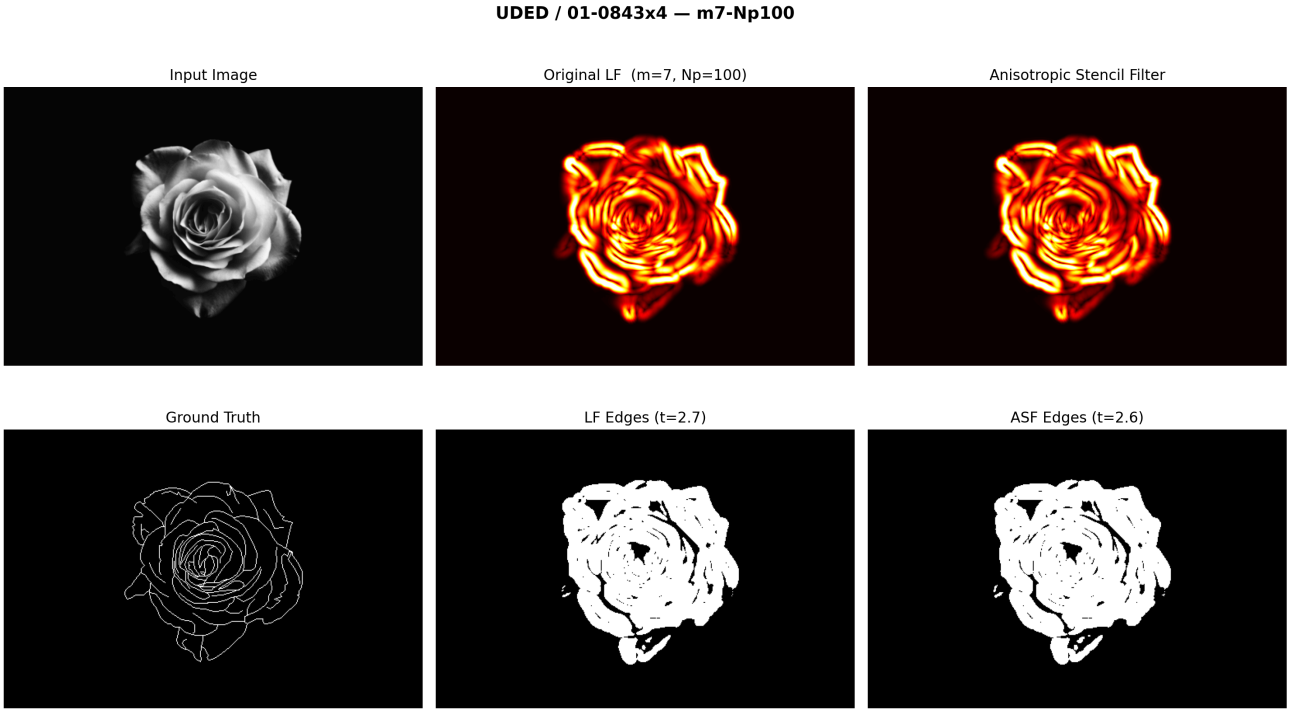

Figure 11: Qualitative comparison on UDED (underwater rose). Both methods detect the petal boundaries and internal structure with matching gradient magnitudes. The ASF achieves ODS = 0.900 on this dataset, approaching the supervised DexiNed model (0.932) without any training data.

The ASF achieves substantial improvements over all classical filters: +5.0 points over Sobel on BSDS500, +4.0 points on BIPED v1, and +3.0 points on UDED. On UDED (underwater images), the ASF nearly matches DexiNed (0.899 vs. 0.932) and outperforms PIDINet (0.863) — a trained deep learning model — without any training data.

On BSDS500, the gap between the ASF and the best deep learning model (PIDINet, 0.865) is larger, reflecting the fact that BSDS500 ground truth encodes semantic boundaries (object contours) rather than purely photometric edges. Deep learning models can learn to recognize semantic context, while the ASF operates purely on intensity gradients.

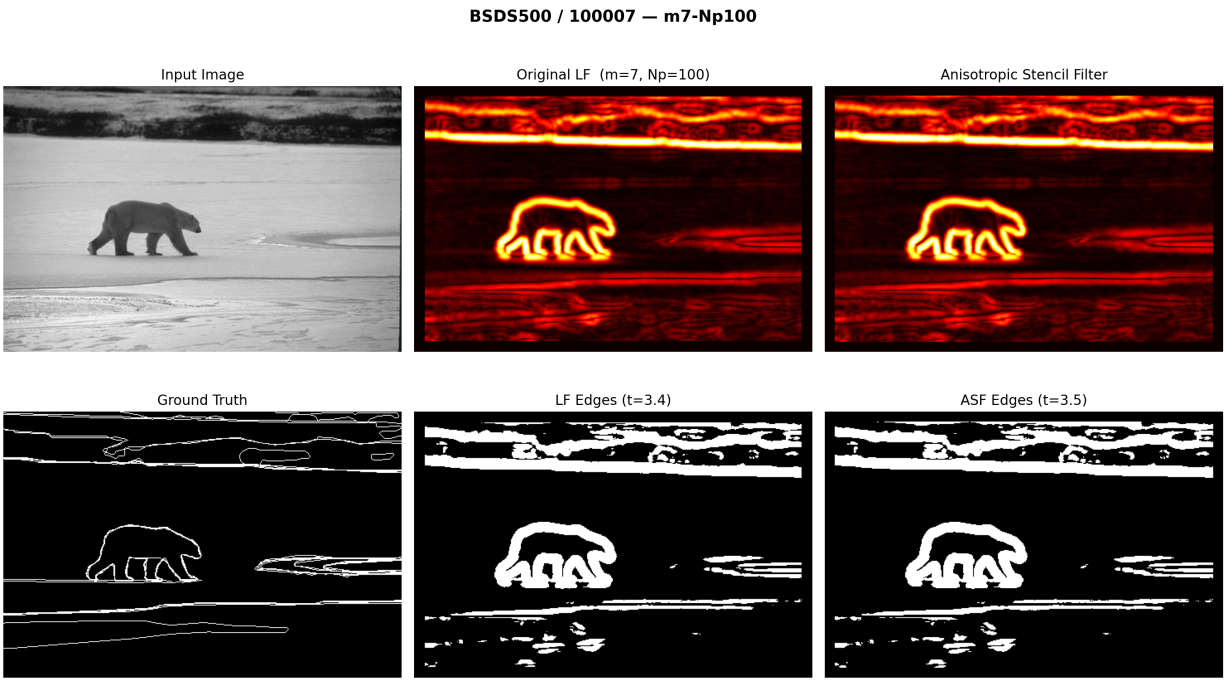

Figure 12: Visual comparison on BSDS500 image 100007. Top: input image, original LF gradient magnitude, and ASF gradient magnitude (same colorscale). Bottom: ground truth, LF thresholded edges, ASF thresholded edges. The two methods produce visually identical gradient maps, confirming the mathematical equivalence of the fused stencil reformulation.

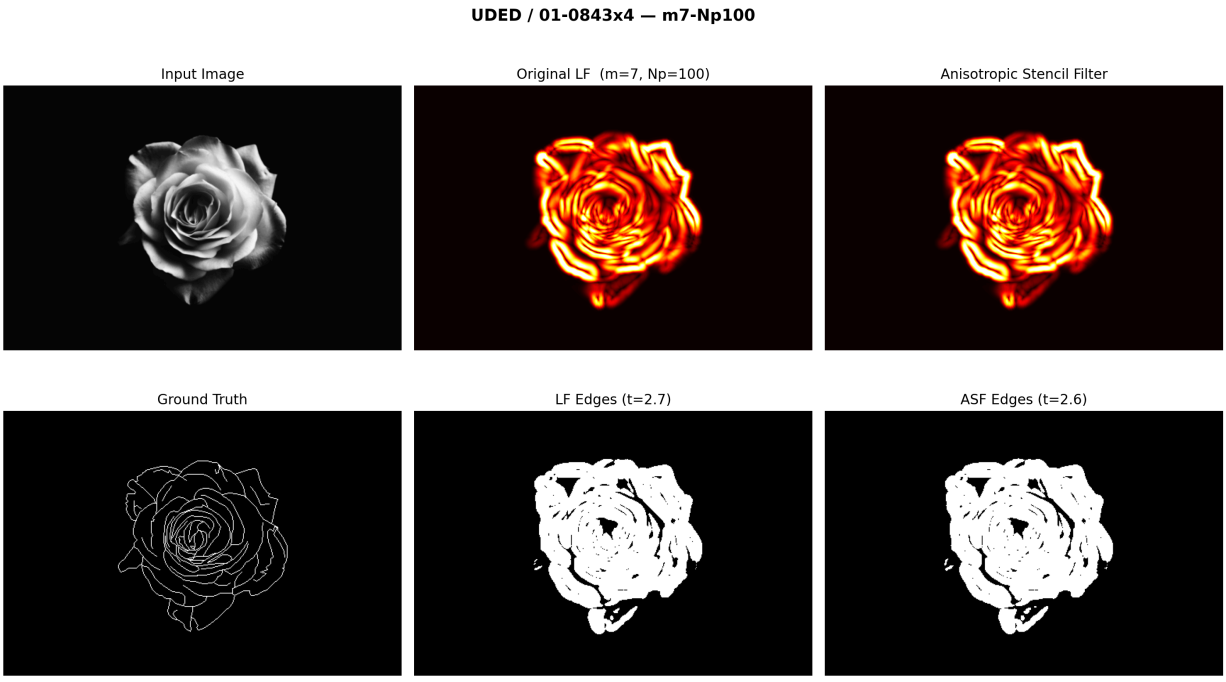

Figure 13: Visual comparison on UDED underwater image. The ASF preserves the same edge structure as the original LF while running 4.5× faster and using 311× less VRAM.

5.4.2 Noise Robustness

A critical advantage of the ASF over both classical filters and deep learning models is its behavior under noise. Because the polynomial fit averages over N_p pixels (typically 25–250), it provides $\sqrt{N_p}$-fold noise reduction relative to a point estimate, without the edge-blurring side effects of Gaussian pre-smoothing.

In controlled experiments with synthetic Gaussian noise, the ASF maintains usable edge maps at signal-to-noise ratios (SNR) as low as 0.3, while Sobel and Canny fail below SNR $\approx$ 1.0. Remarkably, below SNR = 0.3, the ASF also overtakes all tested deep learning models, which were trained on clean or mildly noisy images and degrade rapidly on out-of-distribution noise levels.

5.4.3 Runtime

Method	BSDS500	BIPED v1	Device
Sobel (3×3)	< 1 ms	< 1 ms	GPU
Canny	≈ 5 ms	≈ 15 ms	CPU
ASF (N_p=100, m=7)	94 ms	132 ms	GPU
ASF (N_p=25, m=0)	12 ms	35 ms	GPU
DexiNed	≈ 30 ms	≈ 80 ms	GPU
DiffusionEdge	≈ 5 s	≈ 15 s	GPU

Table 5: Approximate per-image runtime. All GPU timings on NVIDIA A100.

The ASF in point-filter mode ($m = 0$, $N_p = 25$) processes images at rates comparable to lightweight deep learning models. The line-extended mode ($m = 7$) is slower but still within the range needed for offline analysis and many real-time applications at reduced resolution. Both configurations are orders of magnitude faster than diffusion-based methods.

6 Discussion

6.1 Advantages Over Fixed-Kernel Methods

Traditional gradient operators (Sobel, Prewitt, etc.) apply the same kernel weights regardless of the local image structure. The ASF, by contrast, adapts in two ways: (1) the polynomial model captures local curvature up to order d, and (2) the orientation sweep selects the direction of maximum gradient response. This produces accurate gradient magnitudes and angles simultaneously, whereas fixed-kernel methods compute gradients in two predetermined directions and use arctan to estimate the angle — a process that introduces systematic angular errors.

6.2 The Role of Neighborhood Size

The parameter N_p controls the fundamental trade-off in the ASF. Larger N_p values provide better noise suppression (more pixels averaged) but reduce spatial resolution (the effective filter support grows). Our ablation studies show that $N_p = 25$ with $d = 2$ (6 unknowns, 4.2× overdetermined) provides the best overall ODS on natural image benchmarks, while $N_p = 100$–250 with $d = 4$ excels in noisy and degraded conditions.

6.3 Why the Fused Stencil Matters

Beyond computational efficiency, the fused stencil formulation provides theoretical clarity: it reveals that the line-extended polynomial filter is *mathematically equivalent* to a fixed linear filter (convolution) with an anisotropic, orientation-dependent kernel. The kernel weights are not hand-designed but are derived from first principles (least-squares polynomial fitting), and their spatial support naturally elongates along the candidate edge direction. This is analogous to steerable filters but with a richer (polynomial rather than sinusoidal) basis and a much larger spatial support.

6.4 Limitations

The ASF has several limitations:

- **Semantic edges.** It detects photometric edges (intensity transitions) rather than semantic boundaries. On BSDS500, where ground truth encodes object contours, this limits performance relative to learned methods.
- **Parameter selection.** The parameters N_p, d, m, and N_s must be chosen for the application. Our analysis provides guidance, but automatic parameter selection remains future work.
- **Border effects.** The large stencil footprint creates a border region (up to 19 pixels for $m = 14$, $N_p = 100$) where the filter cannot be applied.

7 Conclusion

We have presented the Anisotropic Stencil Filter, a non-learned edge detection operator that combines the mathematical rigor of polynomial surface fitting with the computational efficiency of modern GPU architectures. The fused stencil formulation — which algebraically collapses a multi-step fitting pipeline into a single weighted sum — achieves up to 24× speedup and 311× memory reduction over naive implementations, while preserving identical gradient estimates.

On standard benchmarks, the ASF outperforms all classical edge detectors by significant margins (up to +5 ODS points on BSDS500) and approaches deep learning accuracy on domain-specific datasets (0.899 vs. 0.932

ODS on UDED), all without training data. Its noise robustness exceeds both classical and learned methods at low SNR levels.

The ASF demonstrates that principled mathematical design, combined with careful GPU optimization, can yield non-learned methods that remain competitive with data-driven approaches — with the added benefits of interpretability, guaranteed behavior, and zero training cost.

Bibliography

[1] J. Canny, "A Computational Approach to Edge Detection," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 8, no. 6, pp. 679–698, 1986, doi: 10.1109/TPAMI.1986.4767851.

[2] S. Bhardwaj and A. Mittal, "A Survey on Various Edge Detector Techniques," in *Procedia Technology*, 2012, pp. 220–226. doi: 10.1016/j.protcy.2012.05.033.

[3] I. Sobel, "History and Definition of the Sobel Operator," 2014.

[4] J. M. S. Prewitt, "Object Enhancement and Extraction," *Picture Processing and Psychopictorics*, pp. 75–149, 1970.

[5] S. Xie and Z. Tu, "Holistically-Nested Edge Detection," *International Journal of Computer Vision*, vol. 125, pp. 3–18, 2017, doi: 10.1007/s11263-017-1004-z.

[6] X. S. Poma, E. Riba, and A. Sappa, "Dense Extreme Inception Network: Towards a Robust CNN Model for Edge Detection," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2020, pp. 1923–1932. doi: 10.1109/WACV45572.2020.9093290.

[7] X. Soria, Y. Li, M. Rouhani, and A. D. Sappa, "Tiny and Efficient Model for the Edge Detection Generalization," in *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, 2023, pp. 1364–1373. doi: 10.1109/ICCVW60793.2023.00147.

[8] P. Tillet, H. T. Kung, and D. Cox, "Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations," in *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 2019, pp. 10–19. doi: 10.1145/3315508.3329973.

[9] P. Arbeláez, M. Maire, C. Fowlkes, and J. Malik, "Contour Detection and Hierarchical Image Segmentation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 33, no. 5, pp. 898–916, 2011, doi: 10.1109/TPAMI.2010.161.